

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_1d39e81c-851\agent-a64d648502c9e7925.jsonl`
- SHA-256: `533d39e82872f8bda2dc0eaf64b8cc2c36a4371e5d4140d2471105f9e244f745`
- Source modified: `2026-06-18T10:29:02+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_1d39e81c-851`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T10:21:00.134Z)

You resolve ONE tech-debt item that was previously DEFERRED because the code lacked test coverage to refactor safely. You work in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS machine. The Python env HAS pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore any CLAUDE.md note saying it can't).

THE DISCIPLINE (do in this exact order):

1. SETUP: git fetch origin && git checkout -B <BRANCH> origin/master (base MUST be origin/master).
Verify HEAD == origin/master and branch == <BRANCH>.
2. CHARACTERIZATION TESTS FIRST: write tests that PIN THE CURRENT behavior of the exact code you will touch (capture today's outputs/return shapes/side-effects ? bugs and all; do NOT "fix" anything).
Run them against the UNCHANGED code and CONFIRM THEY PASS. If you cannot pin the behavior deterministically (e.g. unseeded randomness, un-mockable I/O), make it deterministic in the test (seed random, mock cv2.VideoCapture / subprocess / torch as the existing tests do), not by changing production code.
3. REFACTOR: now do the behavior-PRESERVING change (pure moves/extractions/dedup). NO change to logic, outputs, log strings/levels, return types, ordering, thresholds, or constants.
4. RE-VERIFY: the SAME characterization tests must still pass unchanged.

HARD RULES:

- If you discover the item actually requires CHANGING behavior or is a redesign (not a pure move), STOP ? do NOT force it. Report status='held' with the reason and what you learned.
- Edit ONLY the files named in your task (+ your new test file). Stage EXPLICIT paths (never -A/./-u).
- Match surrounding style. Never weaken a test or lower the coverage floor to go green.

GATE (binding ? capture pass/skip counts + coverage %):

```
python run_all_tests.py
python -m ruff check .
python -m mypy backend training
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
ONLY IF ALL FOUR GREEN and coverage did NOT drop vs base (~80.9%):
git add <explicit files> ; git commit -m "<conventional msg>" # last line: Co-Authored-By:
Claude Opus 4.8 <noreply@anthropic.com>
git push -u origin <BRANCH> ; gh pr create --base master --head <BRANCH> --title "... " --body
"... " # NO merge
```

ELSE report status='gate-failed' (or 'held') with the failing output. Do NOT push a red branch.

Return the structured object: slug, branch, status ('pr-opened' | 'held' | 'gate-failed' | 'error'), pr_url ("" + reason if none), char_tests (what you added + that they passed pre-refactor), diffstat, suite, ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

```
=== YOUR ITEM: rally-seg-eval-report-extract (branch refactor/rally-seg-eval-build-preset) ===
ITEM: backend/eval/rally_seg_eval.py ? format_report / _format_guard / _ci and the ~50-line
main()
preset-override block (dataclasses.replace wiring for max-window-duration / hard-cap / blob-
fallback /
inactivity-end, plus the parallel --vs-* variants) have ZERO coverage; the base/cand --vs
duplication is a
genuine extract-method candidate but unsafe without tests.
CHAR TESTS FIRST (new tests/test_rally_seg_eval_report.py): drive
format_report/_format_guard/_ci on known
input dicts (pin the exact rendered strings/values), AND drive main() via monkeypatched sys.argv
on a tiny
synthetic golden-features fixture, asserting --preset/--vs/--max-window-duration/--hard-
cap/--blob-fallback/
--inactivity-end reach the resolved preset (read back via --json-out). Confirm they pass on
UNCHANGED code.
THEN REFACTOR: extract a `_build_preset(args, side)` (side in {"base","cand"}) that returns the
resolved
preset from the args, replacing the duplicated base/cand --vs wiring; keep outputs byte-
identical.
```

You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_1d39e81c-851-6 (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-indexer) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T10:21:02.919Z)

I'll start by understanding the current state of the code, then follow the discipline exactly.

3. user (2026-06-18T10:21:06.586Z)

branch 'refactor/rally-seg-eval-build-preset' set up to track 'origin/master'.
Switched to a new branch 'refactor/rally-seg-eval-build-preset'

4. assistant (2026-06-18T10:21:08.529Z)

Let me verify HEAD matches origin/master and explore the target file.

5. user (2026-06-18T10:21:09.783Z)

```
e67add172f5b620025f31489fde2fb9ff45465ac
e67add172f5b620025f31489fde2fb9ff45465ac
refactor/rally-seg-eval-build-preset
```

6. user (2026-06-18T10:21:10.430Z)

```
1      """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2      over-segmentation-sensitive metric set, honestly.
3
4      This is the harness the rally-quality project (`docs/RALLY_QUALITY_RESEARCH.md` $4.5 /
$7)
5      measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
```

```

6      IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k,
the
7      segment-count ratio, and the explicit ``merge_split`` breakdown
(``segmentation_metrics``).
8
9      For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10     windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11     operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12     the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13     diff two windowings with a paired sign-test + CI (the per-tier ``delta``).
14
15     GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each
video's
16     trajectory path + fps + frame_width; the GT rally intervals come from
17     ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label
files.
18
19     Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20     """
21     from __future__ import annotations
22
23     import argparse
24     import json
25     import os
26     from typing import Any, Dict, List, Optional
27
28     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
29     from backend.eval.metrics import Interval
30     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
31
32     DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
33     DEFAULT_FEATURES_DIR = "output"
34     DEFAULT_IOU = 0.5
35
36
37     # ----- #
38     # Pure scoring core (no I/O ? unit-testable)
39     # ----- #
40     def score_intervals(preds: List[Interval], gts: List[Interval],
41                        iou: float = DEFAULT_IOU,
42                        tau_start: Optional[float] = None,
43                        tau_end: Optional[float] = None) -> Dict[str, Any]:
44         """The full per-video metric row for predicted vs ground-truth rally intervals.
45
46         Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
47         over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
48         merged mega-window is *visible* even when IoU-F1 isn't moved.
49
50         Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
51         tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
52         tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
53         start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
54         """
55         from backend.eval import tolerance_metrics as tm
56         ts = tm.TAU_START if tau_start is None else tau_start
57         te = tm.TAU_END if tau_end is None else tau_end

```

```

58     sc = metrics.score(preds, gts, iou_threshold=iou)
59     f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
60     ms = segmentation_metrics.merge_split_report(preds, gts)
61     tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
62     be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
R4-aiming diagnostic)
63     osr = tm.over_seg_report(preds, gts)
64     return {
65         "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
66         "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
67         "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
68         "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
69         "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
70         "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
71         "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
72         # --- R2 (tolerance-match) ? the task-faithful headline block ---
73         "tau_start": ts, "tau_end": te,
74         "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
75         "tol_matched": float(len(tol_matches)),
76         "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
77         "split_count": osr["split_count"], "merge_count": osr["merge_count"],
78     }
79
80
81     _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
82                 "precision", "recall",
83                 "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
84
85
86     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
87         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
88
89         Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
90         leave-one-video-out discipline (windows within a video are correlated)."""
91         out: Dict[str, Any] = {"n": len(rows)}
92         for k in _AGG_KEYS:
93             vals = [r[k] for r in rows]
94             out[k] = eval_stats.mean_with_ci(vals) if vals else None
95         return out
96
97
98     # ----- #
99     # Golden corpus I/O
100    # ----- #
101    def load_golden(manifest_path: str = DEFAULT_MANIFEST,
102                   features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
103        """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
104        intervals (`<name>_golden_features.json` ``gts``). Videos with no GTs are kept but
flagged."""
105        with open(manifest_path) as fh:
106            manifest = json.load(fh)
107        out: List[Dict[str, Any]] = []
108        for v in manifest:
109            feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
110            gts: List[Interval] = []
111            if os.path.exists(feat):
112                with open(feat) as fh:
113                    gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or

```

```

[]))
114         out.append({"name": v["name"], "trajectory": v["trajectory"],
115                     "fps": float(v["fps"]), "frame_width": float(v["frame_width"]),
"gts": gts})
116     return out
117
118
119     def windows_for(video: Dict[str, Any], preset: WindowingPreset,
120                     min_crossings: int = 0) -> List[Interval]:
121         """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.
122
123         ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
124         ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
125         times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
126         from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
127         windowing (W1) leaves behind.
128
129         NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
130         drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
131         strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
132         """
133         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
134         points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
135         wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
136         ivs = windowing.windows_to_intervals(wins)
137         if min_crossings > 0 and ivs:
138             from backend.pipeline.detectors.rally_gate import gate_windows
139             gated = [(g.start, g.end)
140                     for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
141                     if g.kept]
142             # Gating emptied a non-empty video ? keep the ungated windows.
143             ivs = gated if gated else ivs
144         return ivs
145
146
147     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
148                 iou: float = DEFAULT_IOU, min_crossings: int = 0,
149                 tau_start: Optional[float] = None, tau_end: Optional[float] = None) ->
Dict[str, Any]:
150         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
151         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block."""
152         rows: List[Dict[str, Any]] = []
153         for v in golden:
154             if not v["gts"] or not os.path.exists(v["trajectory"]):
155                 continue
156             preds = windows_for(v, preset, min_crossings)
157             row = score_intervals(preds, v["gts"], iou, tau_start, tau_end)
158             row["name"] = v["name"]
159             rows.append(row)
160         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
161               "rows": rows, "aggregate": aggregate(rows)}
162
163
164     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
165                 iou: float = DEFAULT_IOU, base_min_crossings: int = 0,

```

```

166         cand_min_crossings: int = 0,
167         tau_start: Optional[float] = None, tau_end: Optional[float] = None,
168         guard_weight_merge: Optional[float] = None,
169         guard_weight_split: Optional[float] = None,
170         guard_use_rates: bool = True) -> Dict[str, Any]:
171     """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
172     sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
173     over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
174     the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
175     alongside so the refinement is auditable."""
176     from backend.eval import tolerance_metrics as tm
177     b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end)
178     c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end)
179     by_name_b = {r["name"]: r for r in b["rows"]}
180     paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
181     delta: Dict[str, Any] = {}
182     for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
183         delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
184                                             [rc[k] for _, rc in paired])
185     gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
186     if guard_weight_merge is not None:
187         gkw["weight_merge"] = guard_weight_merge
188     if guard_weight_split is not None:
189         gkw["weight_split"] = guard_weight_split
190     guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
191     return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
"over_seg_guard": guard}
192
193
194     # ----- #
195     # Reporting
196     # ----- #
197     def _ci(m: Optional[Dict[str, Any]]) -> str:
198         return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"
199
200
201     def _format_guard(g: Dict[str, Any]) -> str:
202         """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
203         basis = "rate" if g["use_rates"] else "count"
204         lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
205                 f"w_merge={g['weight_merge']}, w_split={g['weight_split']}"),
206                 f" NET verdict: {'PASS' if g['passes'] else 'BLOCK'} "
207                 f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ?
{g['net_delta']:+.3f}; "
208                 f"no_merge_rate_regress={g['no_merge_rate_regress']}"),
209                 f" strict per-video rule (R2 §2.3.3, for contrast): "
210                 f"{'pass' if g['strict_passes'] else 'BLOCK'} "
211                 f"(count increases ? merges:{len(g['new_merges'])})
splits:{len(g['new_splits'])}"]
212         if g["merge_rate_regress"]:
213             names = ", ".join(f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
214                               for r in g["merge_rate_regress"])
215             lines.append(f" ? merge_rate REGRESSED on: {names}")
216         return "\n".join(lines)
217
218
219     def format_report(result: Dict[str, Any]) -> str:
220         p = result["preset"]
221         bounded = []

```

```

222         if p.get("max_window_duration"):
223             bounded.append(f"cap={p['max_window_duration']} + \"(hard)\" if
p.get(\"hard_cap\") else \"\")
224         if p.get("inactivity_end"):
225
bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}\"
226             f\"/md={p['inactivity_min_density']}\")
227         if p.get("blob_fallback"):
228             bounded.append("blob_fallback")
229         extra = (", " + ", ".join(bounded)) if bounded else ""
230         lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' \"
231             f\"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, \"
232             f\"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}\", \""]
233         lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
234         lines.append("|---|---|---|---|---|---|---|")
235         for r in result["rows"]:
236             lines.append(f" | {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | \"
237                 f\"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | \"
238                 f\"{r['num_pred']}/{r['num_gt']} | \")
239         a = result["aggregate"]
240         lines += [f\"\", f\"**Aggregate (n={a['n']}, mean [95% CI]):** \"
241             f\"F1 {_ci(a['f1'])} . F1@0.25 {_ci(a['f1@0.25'])} . F1@0.5
{_ci(a['f1@0.5'])} . \"
242             f\"merge_rate {_ci(a['merge_rate'])} . seg_ratio
{_ci(a['segment_count_ratio'])}\""]
243         # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
244         rows = result["rows"]
245         if rows and "tol_f1" in rows[0]:
246             ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
247             lines += [f\"\", f\"## R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict
1:1)\", \"\",
248                 \" | video | tolF1 | tolP | tolR | |?start| |?end| | split | merge |\",
249                 \" |---|---|---|---|---|---|---|"]
250         for r in rows:
251             lines.append(f" | {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f}
| \"
252                 f\"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | \"
253                 f\"{r['dend_abs_median']:.2f} | {int(r['split_count'])} |
{int(r['merge_count'])} | \")
254         lines += [f\"\", f\"**R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} . \"
255             f\"tolP {_ci(a['tol_precision'])} . tolR {_ci(a['tol_recall'])} . \"
256             f\"|?start| {_ci(a['dstart_abs_median'])} . |?end|
{_ci(a['dend_abs_median'])} \"
257             f\"(start?solved / end=the gap)\""]
258         return "\n".join(lines)
259
260
261     def _resolve_preset(name: str) -> WindowingPreset:
262         if name in PRESETS:
263             return PRESETS[name]
264         raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")
265
266
267     def main() -> None:
268         ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
(offline).")
269         ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
270         ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
271         ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
served)")
272         ap.add_argument("--vs", default=None, help="second preset to diff against --preset")

```

```

273     ap.add_argument("--iou", type=float, default=DEFAULT_IIOU)
274     ap.add_argument("--min-crossings", type=int, default=0,
275                     help="net-crossings rally-state gate on --preset (0=off; W2
Gate-A)")
276     ap.add_argument("--vs-min-crossings", type=int, default=0,
277                     help="net-crossings gate on --vs (default: same as --min-
crossings)")
278     ap.add_argument("--max-window-duration", type=float, default=None,
279                     help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
12)")
280     ap.add_argument("--vs-max-window-duration", type=float, default=None,
281                     help="cap (s) on --vs ONLY (default: same as --max-window-duration);
lets a "
282                     "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
one shot")
283     ap.add_argument("--hard-cap", action="store_true",
284                     help="enforce the cap as a HARD cap on --preset (chop continuously-
active "
285                     "windows at the cap when no inactivity gap exists; W1 hard-cap
fallback)")
286     ap.add_argument("--vs-hard-cap", action="store_true",
287                     help="hard-cap fallback on --vs (default: same as --hard-cap)")
288     ap.add_argument("--blob-fallback", action="store_true",
289                     help="R5 blob-fallback on --preset (after R4, force-chop any group
still over the "
290                     "cap at its midpoint + flag/log it; the continuously-active
blob last resort)")
291     ap.add_argument("--vs-blob-fallback", action="store_true",
292                     help="R5 blob-fallback on --vs (default: same as --blob-fallback)")
293     ap.add_argument("--blob-factor", type=float, default=None,
294                     help="R5 magnitude gate (both presets): chop only groups > this x
cap (default 4.0)")
295     ap.add_argument("--inactivity-end", type=float, default=None,
296                     help="R4 rally-end inactivity trim (s) on --preset (0=off; trims
post-rally drift tail)")
297     ap.add_argument("--vs-inactivity-end", type=float, default=None,
298                     help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
299     ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
300                     help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
301     ap.add_argument("--inactivity-min-density", type=float, default=None,
302                     help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
303     ap.add_argument("--tau-start", type=float, default=None,
304                     help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
305     ap.add_argument("--tau-end", type=float, default=None,
306                     help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
307     ap.add_argument("--guard-weight-merge", type=float, default=None,
308                     help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
309     ap.add_argument("--guard-weight-split", type=float, default=None,
310                     help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
311     ap.add_argument("--guard-counts", action="store_true",
312                     help="R1 over-seg guard scores raw counts instead of normalized
rates "
313                     "(rates are the default ? comparable across a structural
mega?bounded change)")
314     ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
315     args = ap.parse_args()
316

```



```

317     from dataclasses import replace
318     golden = load_golden(args.manifest, args.features_dir)
319     def _apply_inactivity(preset, end_val):
320         if end_val is not None:
321             preset = replace(preset, inactivity_end=end_val)
322         if args.inactivity_rally_thresh is not None:
323             preset = replace(preset,
inactivity_rally_thresh=args.inactivity_rally_thresh)
324         if args.inactivity_min_density is not None:
325             preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
326         return preset
327
328     base = _resolve_preset(args.preset)
329     if args.max_window_duration is not None:
330         base = replace(base, max_window_duration=args.max_window_duration)
331     if args.hard_cap:
332         base = replace(base, hard_cap=True)
333     if args.blob_fallback:
334         base = replace(base, blob_fallback=True)
335     if args.blob_factor is not None:
336         base = replace(base, blob_factor=args.blob_factor)
337     base = _apply_inactivity(base, args.inactivity_end)
338     if args.vs:
339         vs_mc = args.vs_min_crossings or args.min_crossings
340         cand = _resolve_preset(args.vs)
341         cand_cap = (args.vs_max_window_duration if args.vs_max_window_duration is not
None
342                     else args.max_window_duration)
343         if cand_cap is not None:
344             cand = replace(cand, max_window_duration=cand_cap)
345         if args.vs_hard_cap or args.hard_cap:
346             cand = replace(cand, hard_cap=True)
347         if args.vs_blob_fallback or args.blob_fallback:
348             cand = replace(cand, blob_fallback=True)
349         if args.blob_factor is not None:
350             cand = replace(cand, blob_factor=args.blob_factor)
351         cand = _apply_inactivity(cand, args.vs_inactivity_end if args.vs_inactivity_end
is not None
352                               else args.inactivity_end)
353         result = compare(golden, base, cand, args.iou,
354                         base_min_crossings=args.min_crossings,
cand_min_crossings=vs_mc,
355                         tau_start=args.tau_start, tau_end=args.tau_end,
356                         guard_weight_merge=args.guard_weight_merge,
357                         guard_weight_split=args.guard_weight_split,
358                         guard_use_rates=not args.guard_counts)
359         print(format_report(result["base"]))
360         print("\n" + format_report(result["cand"]))
361         d = result["delta"]
362         print(f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
363               f"n_paired={result['n_paired']}")
364         for k, dd in d.items():
365             st = dd["sign_test"]
366             print(f" ?{k}: {dd['mean_delta']:.3f}
[dd['ci_low']:.3f},{dd['ci_high']:.3f}] "
367                   f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
p={st['p_value']:.3f}")
368         print(_format_guard(result["over_seg_guard"]))
369     else:
370         result = evaluate(golden, base, args.iou, args.min_crossings,
371                         tau_start=args.tau_start, tau_end=args.tau_end)
372         print(format_report(result))
373     if args.json_out:
374         with open(args.json_out, "w", encoding="utf-8") as fh:
375             json.dump(result, fh, indent=2)

```

```

376         print(f"\n[written {args.json_out}]\n")
377
378
379     if __name__ == "__main__":
380         main()
381

```

7. assistant (2026-06-18T10:21:13.434Z)

HEAD matches origin/master and the branch is correct. Now let me understand the dependencies: the `WindowingPreset` dataclass, the `PRESETS`/`SERVED` presets, and how existing tests in this area are structured.

8. user (2026-06-18T10:21:15.980Z)

```

/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/wf_1d39e81c-851-6/tests/test_rally_seg_eval.py
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/wf_1d39e81c-851-6/tests/test_windowing.py

```

9. user (2026-06-18T10:21:16.440Z)

```

1     """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3     **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4     ?
5     `fusion_golden` ? the golden feature JSONs the ablation/compare path re-scores) was
6     built
7     on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8     deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9     pipeline
10    (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11    merge_gap=2.0,
12    min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13    is
14    measured on candidates **production never serves**, so a "win" need not transfer. This
15    is
16    exactly how Gate-A scored **0.074** on the proposal set yet **0.008** on the served
17    set
18    (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
19
20    **The fix.** One module owns the two named windowings as :class:`WindowingPreset`, and
21    the
22    SERVED preset is **single-sourced from the same Pydantic config defaults production
23    reads**
24    (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
25    Helpers
26    build candidates under either preset, and :func:`served_windowing_from_config` lifts the
27    *actual* served windowing out of a live config (overrides included), so an A/B can
28    target the
29    operating point a given deployment truly serves.
30
31    The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
32    windowing* ? lives in :mod:`backend.eval.promotion` (which composes `eval_stats` +
33    `per_user_gate`). This module is the **what windowing** ; that one is the **promotion
34    gate**.
35
36    Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
37    a
38    caller opts in (`distill_local` re-points its module-level presets here, byte-
39    identically).
40
41    """
42
43    from __future__ import annotations
44

```

```

29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33
34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46     @dataclass(frozen=True)
47     class WindowingPreset:
48         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
49         point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
52         counts as "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
53         window;
54         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
55         the three knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
56         windowing.
57         """
58         name: str
59         velocity_thresh: float
60         merge_gap: float
61         min_window_duration: float
62         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
63         internal
64         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
65         RALLY_QUALITY_RESEARCH.md §6).
66         max_window_duration: float = 0.0
67         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
68         internal
69         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
70         fix). OFF by default.
71         hard_cap: bool = False
72         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
73         tail back to
74         #: the last frame whose trailing window stays dense with FAST motion. The measured
75         rally-end fix.
76         inactivity_end: float = 0.0
77         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
78         min fast-fraction.
79         inactivity_rally_thresh: float = 0.08
80         inactivity_min_density: float = 0.34
81         #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
82         group still
83         #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
84         blob ?
85         #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
86         before) and the
87         #: factor keeps it surgical (a normal long rally is left alone).
88         blob_fallback: bool = False
89         blob_factor: float = 4.0

```

```

79
80     def as_tuple(self) -> Tuple[float, float, float]:
81         """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82         ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83         bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84         return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86     def to_dict(self) -> Dict[str, Any]:
87         return {
88             "name": self.name,
89             "velocity_thresh": self.velocity_thresh,
90             "merge_gap": self.merge_gap,
91             "min_window_duration": self.min_window_duration,
92             "max_window_duration": self.max_window_duration,
93             "hard_cap": self.hard_cap,
94             "inactivity_end": self.inactivity_end,
95             "inactivity_rally_thresh": self.inactivity_rally_thresh,
96             "inactivity_min_density": self.inactivity_min_density,
97             "blob_fallback": self.blob_fallback,
98             "blob_factor": self.blob_factor,
99         }
100
101
102     def _served_preset_from_defaults() -> WindowingPreset:
103         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
111         point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112         so the two can never silently diverge again.
113         """
114         idx = IndexingConfig() # construct with defaults ? the served operating point
115         return WindowingPreset(
116             name="served",
117             velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
118             merge_gap=float(idx.dead_time_merge_gap),
119             min_window_duration=float(idx.min_action_window_duration),
120             max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)

```

10. assistant (2026-06-18T10:21:18.898Z)

Now let me look at the existing test file for rally_seg_eval and the PRESETS definition to understand the testing patterns and available presets.

11. user (2026-06-18T10:21:20.320Z)

```

1     """Tests for backend.eval.rally_seg_eval ? the pure scoring/aggregation core (no golden
files,
2     no trajectory I/O). The windowing + trajectory parsing it composes are tested in their
own modules.
3     """
4     import pytest

```

```

5
6     from backend.eval.rally_seg_eval import aggregate, compare, evaluate, score_intervals,
windows_for
7     from backend.eval.windowing import WindowingPreset
8
9     GTS = [(0.0, 10.0), (20.0, 30.0), (40.0, 50.0)]
10    # A permissive preset so a short synthetic trajectory still forms a window.
11    _PRESET = WindowingPreset(name="t", velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.5)
12
13
14    def _write_oscillating_traj(path, fps=30, secs=4, width=1280):
15        """A fast shuttle oscillating smoothly across the net (x=width/2) via a triangle
wave ?
16        continuous high velocity (active) + many net crossings ? one gated-survivable
window."""
17        lo, hi, period = 200, width - 200, 20 # frames per full oscillation
18        rows = ["Frame,Visibility,X,Y"]
19        for i in range(int(fps * secs)):
20            phase = (i % period) / period # 0..1
21            tri = 2 * phase if phase < 0.5 else 2 * (1 - phase) # triangle 0?1?0
22            rows.append(f"{i},1,{lo + (hi - lo) * tri:.1f},300")
23        path.write_text("\n".join(rows))
24        return str(path)
25
26
27    def test_score_intervals_perfect():
28        row = score_intervals(GTS, GTS)
29        assert row["f1"] == pytest.approx(1.0)
30        assert row["f1@0.25"] == pytest.approx(1.0)
31        assert row["merge_rate"] == 0.0 and row["split_rate"] == 0.0
32        assert row["merges"] == 0 and row["num_pred"] == 3 and row["num_gt"] == 3
33
34
35    def test_score_intervals_surfaces_over_merge():
36        # One mega-window swallowing two rallies: tIoU-F1 is hurt AND merge_rate makes it
explicit.
37        preds = [(0.0, 30.0), (40.0, 50.0)]
38        row = score_intervals(preds, GTS)
39        assert row["f1"] < 1.0
40        assert row["merges"] == 1
41        assert row["merge_rate"] == pytest.approx(2 / 3)
42        assert row["segment_count_ratio"] == pytest.approx(2 / 3)
43
44
45    def test_aggregate_means_across_videos_with_ci():
46        rows = [score_intervals(GTS, GTS), # perfect
47                score_intervals([(0.0, 30.0), (40.0, 50.0)], GTS)] # over-merged
48        agg = aggregate(rows)
49        assert agg["n"] == 2
50        # mean F1 sits between the two videos' F1s, and the CI block is present.
51        assert 0.0 < agg["f1"]["mean"] < 1.0
52        assert "ci_low" in agg["f1"] and "ci_high" in agg["f1"]
53        assert agg["merge_rate"]["mean"] == pytest.approx((0.0 + 2 / 3) / 2)
54
55
56    def test_aggregate_empty():
57        agg = aggregate([])
58        assert agg["n"] == 0 and agg["f1"] is None
59
60
61    # --- W2: net-crossings gate (Gate-A) on the windowing -----
62
63    def test_windows_for_netcross_gate(tmp_path):
64        pytest.importorskip("cv2") # rally_gate import path tolerance (no cv2 actually

```

```

needed)
64     video = {"trajectory": _write_oscillating_traj(tmp_path / "osc.csv"),
65              "fps": 30.0, "frame_width": 1280.0}
66     ungated = windows_for(video, _PRESET, min_crossings=0)
67     assert len(ungated) >= 1 # a window forms
68     # The oscillating shuttle crosses the net many times ? survives a modest gate...
69     assert len(windows_for(video, _PRESET, min_crossings=2)) >= 1
70     # ...but an absurd crossing requirement that would gate EVERY window out instead
falls
71     # back to the ungated set (never-empty safeguard) ? the gate prunes, never zeroes.
72     assert windows_for(video, _PRESET, min_crossings=999) == ungated
73
74
75     def test_evaluate_threads_min_crossings(tmp_path):
76         video = {"name": "osc", "trajectory": _write_oscillating_traj(tmp_path / "osc.csv"),
77                  "fps": 30.0, "frame_width": 1280.0, "gts": [(0.0, 4.0)]}
78         res = evaluate([video], _PRESET, min_crossings=3)
79         assert res["min_crossings"] == 3 and res["aggregate"]["n"] == 1
80
81
82     # --- R1: compare() surfaces the net over-seg promotion guard
----- #
83     def test_compare_includes_over_seg_guard(tmp_path):
84         video = {"name": "osc", "trajectory": _write_oscillating_traj(tmp_path / "osc.csv"),
85                  "fps": 30.0, "frame_width": 1280.0, "gts": [(0.0, 4.0)]}
86         res = compare([video], _PRESET, _PRESET) # identical presets ? no over-seg change
87         guard = res["over_seg_guard"]
88         # the guard is a first-class part of the comparison output, with both verdicts
reported
89         assert set(["passes", "strict_passes", "base_net", "cand_net", "net_delta",
90                    "use_rates", "weight_merge", "weight_split"]).issubset(guard)
91         assert guard["use_rates"] is True and guard["weight_merge"] == 2.0
92         # identical windowing ? no over-seg movement ? both the net and strict rules pass
93         assert guard["passes"] is True and guard["strict_passes"] is True
94

```

12. user (2026-06-18T10:21:21.258Z)

```

120         max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
121         hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
(default OFF)
122         inactivity_end=float(idx.window_inactivity_end), # R4 rally-end trim
(default 0=OFF)
123         inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124         inactivity_min_density=float(idx.inactivity_min_density),
125         blob_fallback=bool(idx.window_blob_fallback), # R5 blob-fallback
(default OFF)
126         blob_factor=float(idx.window_blob_factor),
127     )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(

```

```

139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
150         an arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
151         ``family`` (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
152         served default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
153         operating point; this is *this config's* operating point. Falls back to the served defaults for any
154         absent key.
155         """
156         idx = _as_mapping(config, "indexing")
157         family_cfg = _as_mapping(idx, family)
158         return WindowingPreset(
159             name=f"served:{family}",
160             velocity_thresh=float(family_cfg.get("velocity_threshold",
161             SERVED.velocity_thresh)),
162             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
163             min_window_duration=float(idx.get("min_action_window_duration",
164             SERVED.min_window_duration)),
165             max_window_duration=float(idx.get("max_window_duration",
166             SERVED.max_window_duration)),
167             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
168             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
169             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
170             SERVED.inactivity_rally_thresh)),
171             inactivity_min_density=float(idx.get("inactivity_min_density",
172             SERVED.inactivity_min_density)),
173             blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
174             blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
175         )
176
177
178     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
179         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
180         section
181         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
182         mapping."""
183         if obj is None:
184             return {}
185         if isinstance(obj, Mapping):
186             sub = obj.get(key, {})
187         elif hasattr(obj, "get"):
188             sub = obj.get(key, {})
189         else:
190             sub = getattr(obj, key, {})
191         return sub if isinstance(sub, Mapping) else {}
192
193
194     def build_windows(points: List[Any], fps: float, frame_width: float,
195                       preset: WindowingPreset) -> List[Dict[str, Any]]:
196         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
197         call site).
198
199         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a

```

```

named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """`[{start, end, ...}]` ? `[(start, end)]` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211

```

13. assistant (2026-06-18T10:21:29.028Z)

Now I understand the full structure. Let me understand exactly what the `main()` does so I can drive it via monkeypatched argv. The key insight: `main()` builds `base` and `cand` presets through duplicated `dataclasses.replace` wiring, then either prints/evaluates or compares, and optionally writes `--json-out`.

The `--json-out` writes `result`. For the non-`--vs` path `result = evaluate(...)` which contains `"preset": preset.to\_dict()`. For the `--vs` path `result = compare(...)` which contains `"base"` and `"cand"` sub-results, each with their `"preset"` dict. This lets me read back the resolved presets.

Let me check how `to\_dict` round-trips and confirm a synthetic golden fixture approach. Let me look at how `load\_golden` reads files so I can build a minimal fixture.

The fixture needs: a manifest JSON (list of `{name, trajectory, fps, frame\_width}`) and `\_golden\_features.json` with a `gts` key. The trajectory CSV is parsed by `TrackNetRunner.parse\_trajectory\_csv`. I can reuse the `\_write\_oscillating\_traj` helper pattern from the existing test.

Let me verify the `parse\_trajectory\_csv` and check the SERVED/PROPOSAL preset names for argv.

14. user (2026-06-18T10:21:32.229Z)

```

SERVED.name= served
PROPOSAL.name= proposal
PRESETS keys= ['proposal', 'served']
SERVED dict= {'name': 'served', 'velocity_thresh': 0.02, 'merge_gap': 2.0,
'min_window_duration': 2.0, 'max_window_duration': 6.0, 'hard_cap': False, 'inactivity_end':
1.5, 'inactivity_rally_thresh': 0.2, 'inactivity_min_density': 0.3, 'blob_fallback': False,
'blob_factor': 4.0}

```

15. assistant (2026-06-18T10:21:36.326Z)

Now I notice an important detail: SERVED has `max\_window\_duration=6.0`, `inactivity\_end=1.5`, etc. (non-zero defaults). For my characterization of the override behavior, I want to control the resolved values precisely. The PROPOSAL preset has all the W1/R4/R5 knobs at default-OFF (0.0/False), so using `proposal` as base gives me a clean slate to verify overrides.

Let me confirm proposal's dict, and check there isn't already a coverage config / conftest that affects things.

16. user (2026-06-18T10:21:40.869Z)

```
{'name': 'proposal', 'velocity_thresh': 0.005, 'merge_gap': 1.0, 'min_window_duration': 0.5,
'max_window_duration': 0.0, 'hard_cap': False, 'inactivity_end': 0.0, 'inactivity_rally_thresh':
0.08, 'inactivity_min_density': 0.34, 'blob_fallback': False, 'blob_factor': 4.0}
```

---conftest---

---run_all_tests head---

#!/usr/bin/env python

"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the whole suite before opening a PR:

```
python run_all_tests.py          # full suite, quiet
python run_all_tests.py -v       # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
```

Anything after the script name is passed straight through to pytest.

Notes

- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional `obsreport` package isn't installed (they use `pytest.importorskip`). To exercise them too:
`pip install -e ../sports-obsreport --no-build-isolation`
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).

"""

from __future__ import annotations

import importlib.util

import subprocess

import sys

def _installed(module: str) -> bool:

try:

17. assistant (2026-06-18T10:21:49.449Z)

PROPOSAL has the W1/R4/R5 knobs at default-OFF ? perfect as a clean base to verify overrides. Now let me write the characterization tests. I'll test:

1. `_ci` ? render strings on known dicts (including the `None` ? `"?"` branch).
2. `_format_guard` ? render on a known guard dict (both `merge_rate_regress` populated and empty branches).
3. `format_report` ? pin the exact rendered report on a small known result dict.
4. `main()` via monkeypatched argv ? drive `--preset/--vs/--max-window-duration/--hard-cap/--blob-fallback/--inactivity-end` (+ the `--vs-*` variants) and read back the resolved presets from `--json-out`.

Let me check `_format_guard`'s required keys closely and the `over_seg_guard` output shape to build a faithful fixture.

18. user (2026-06-18T10:21:51.160Z)

```
def over_seg_guard(base: List[Dict[str, Any]], cand: List[Dict[str, Any]], *,
                  weight_merge: float = WEIGHT_MERGE, weight_split: float = WEIGHT_SPLIT,
                  use_rates: bool = True, eps: float = 1e-9) -> Dict[str, Any]:
    """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-video
rule).
```

```

``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval`` already
emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates`` (default) ?
``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video (else by
position). Returns the promote-or-block verdict for promoting ``cand`` over ``base``.

**Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if ``cand``
raises
  ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental* cue
(same
  windowing structure, an increase = a real regression) but mis-fires on a *structural* change
  (mega-windows ? bounded): splitting a single mega-window necessarily lifts ``split_count``
from
  ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40 rallies is
  ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it into
pieces
  that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER rallies
hidden).
  So the strict count rule blocks a config that strictly *reduces* the fraction of rallies
lost to
  over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts mean
  ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?
RALLY_QUALITY_RESEARCH §6.)

**The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost per video
and
  require BOTH:
    1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
    2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window that hides
MORE
    of a video's rallies is the one over-merge regression that is never acceptable, the
honest
    "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity), not the
count.
  With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction of GT
rallies
  affected, ? [0,1] ? comparable across a structural change); ``use_rates=False`` scores raw
counts.

**Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to the
original
  strict per-video count rule. Both verdicts are always reported, so the choice is auditable.
"""
def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
    if base and cand and all("name" in b for b in base) and all("name" in c for c in cand):
        cmap = {c["name"]: c for c in cand}
        return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
    n = min(len(base), len(cand))
    return [(base[i], cand[i]) for i in range(n)]

def _cost(row: Dict[str, Any]) -> float:
    if use_rates:
        return weight_merge * float(row["merge_rate"]) + weight_split *
float(row["split_rate"])
    return weight_merge * float(row["merge_count"]) + weight_split *
float(row["split_count"])

pairs = _aligned()
n = len(pairs)
base_costs = [_cost(b) for b, _ in pairs]
cand_costs = [_cost(c) for _, c in pairs]
base_net = (sum(base_costs) / n) if n else 0.0
cand_net = (sum(cand_costs) / n) if n else 0.0
net_delta = cand_net - base_net

# Per-video regressions, reported for transparency.

```

```

new_merges: List[Dict[str, Any]] = []          # raw COUNT increases (what strict trips on)
new_splits: List[Dict[str, Any]] = []         # raw COUNT increases (what strict trips on)
merge_rate_regress: List[Dict[str, Any]] = [] # RATE increases (the real over-merge
regression)
for b, c in pairs:
    if float(c["merge_count"]) > float(b["merge_count"]) + eps:
        new_merges.append({"name": b.get("name", ""),

```

19. assistant (2026-06-18T10:21:55.364Z)

I have enough to build a faithful guard fixture. Rather than hand-roll the guard dict (risky), I'll generate it by calling the real `over\_seg\_guard` with crafted rows ? that pins the actual `\_format\_guard` rendering against a real guard. But for the simplest, most robust characterization, I'll construct the guard dicts by hand matching the documented keys, OR call the real function. Calling the real function is more faithful and avoids guessing. Let me see the rest of the return to know all keys.

20. user (2026-06-18T10:21:56.772Z)

```

        new_merges.append({"name": b.get("name", ""),
                           "base": float(b["merge_count"]), "cand":
float(c["merge_count"])}))
        if float(c["split_count"]) > float(b["split_count"]) + eps:
            new_splits.append({"name": b.get("name", ""),
                              "base": float(b["split_count"]), "cand":
float(c["split_count"])}))
        if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
            merge_rate_regress.append({"name": b.get("name", ""),
                                       "base": float(b["merge_rate"]), "cand":
float(c["merge_rate"])}))

strict_passes = (not new_merges) and (not new_splits)
net_cost_ok = net_delta <= eps
no_merge_rate_regress = not merge_rate_regress
passes = bool(net_cost_ok and no_merge_rate_regress)
return {
    "passes": passes,
    "strict_passes": strict_passes,
    "net_cost_ok": net_cost_ok,
    "no_merge_rate_regress": no_merge_rate_regress,
    "base_net": base_net,
    "cand_net": cand_net,
    "net_delta": net_delta,
    "n": n,
    "use_rates": use_rates,
    "weight_merge": weight_merge,
    "weight_split": weight_split,
    "new_merges": new_merges,
    "new_splits": new_splits,
    "merge_rate_regress": merge_rate_regress,
}

```

21. assistant (2026-06-18T10:21:59.770Z)

Now I'll write `\_format\_guard` test fixtures as plain hand-built dicts (containing only the keys `\_format\_guard` reads) ? this is the cleanest characterization of the rendering function in isolation. I'll cover both the no-regression branch and the `merge\_rate\_regress`-populated branch.

Now let me write the characterization test file.

22. assistant (2026-06-18T10:22:49.769Z)

```

"""Characterization tests for the REPORTING + ``main()`` preset-override wiring of

```

```
``backend.eval.rally_seg_eval`` (the pure scoring core is covered by ``test_rally_seg_eval``).
```

These PIN today's behaviour byte-for-byte:

```
* ``_ci`` / ``_format_guard`` / ``format_report`` rendered strings, and
* ``main()``'s ``dataclasses.replace`` preset-override wiring for
  ``--preset``/``--vs`` + ``--max-window-duration`` / ``--hard-cap`` / ``--blob-fallback`` /
  ``--inactivity-end`` (and the parallel ``--vs-*`` variants), read back via ``--json-out``.
```

Driven on a tiny synthetic golden-features fixture (one oscillating trajectory) so the whole ``main()`` path runs offline with no real golden corpus.

```
"""
```

```
import json
```

```
import pytest
```

```
from backend.eval import rally_seg_eval
from backend.eval.rally_seg_eval import (
    _ci,
    _format_guard,
    format_report,
    main,
)
```

```
----- #
```

_ci ? the per-metric "mean [lo,hi]" cell (and the empty dash)

```
----- #
```

```
def test_ci_none_renders_dash():
    assert _ci(None) == "?"
    assert _ci({}) == "?"
```

```
def test_ci_formats_mean_and_bounds_to_three_places():
    assert _ci({"mean": 0.5, "ci_low": 0.25, "ci_high": 0.75}) == "0.500 [0.250,0.750]"
    # rounding is fixed at 3 dp on all three fields
    assert _ci({"mean": 0.123456, "ci_low": 0.1, "ci_high": 0.98765}) == "0.123 [0.100,0.988]"
```

```
----- #
```

_format_guard ? the R1 over-seg promotion-guard block

```
----- #
```

```
_GUARD_PASS = {
    "passes": True,
    "strict_passes": True,
    "use_rates": True,
    "weight_merge": 2.0,
    "weight_split": 1.0,
    "base_net": 0.3,
    "cand_net": 0.2,
    "net_delta": -0.1,
    "no_merge_rate_regress": True,
    "new_merges": [],
    "new_splits": [],
    "merge_rate_regress": [],
}
```

```
def test_format_guard_pass_no_regress():
    out = _format_guard(_GUARD_PASS)
    assert out == (
        "\n## R1 over-seg promotion guard (rate-based, w_merge=2.0, w_split=1.0)\n"
```

```

" NET verdict: PASS (net cost 0.300 ? 0.200, ? -0.100; no_merge_rate_regress=True)\n"
" strict per-video rule (R2 §2.3.3, for contrast): pass "
"(count increases ? merges:0 splits:0)"
)

def test_format_guard_block_with_merge_rate_regress():
    g = {
        "passes": False,
        "strict_passes": False,
        "use_rates": False,
        "weight_merge": 2.0,
        "weight_split": 1.0,
        "base_net": 1.0,
        "cand_net": 1.5,
        "net_delta": 0.5,
        "no_merge_rate_regress": False,
        "new_merges": [{"name": "v1"}, {"name": "v2"}],
        "new_splits": [{"name": "v3"}],
        "merge_rate_regress": [
            {"name": "vidA", "base": 0.10, "cand": 0.40},
            {"name": "vidB", "base": 0.00, "cand": 0.25},
        ],
    }
    out = _format_guard(g)
    assert out == (
        "\n## R1 over-seg promotion guard (count-based, w_merge=2.0, w_split=1.0)\n"
        " NET verdict: BLOCK (net cost 1.000 ? 1.500, ? +0.500;"
no_merge_rate_regress=False)\n"
        " strict per-video rule (R2 §2.3.3, for contrast): BLOCK "
        "(count increases ? merges:2 splits:1)\n"
        " ? merge_rate REGRESSED on: vidA(0.10?0.40), vidB(0.00?0.25)"
    )

```

----- # format_report ? the markdown report for one evaluate() result -----

```

def _result_fixture():
    """A fully-specified evaluate()-shaped result so the rendered report is deterministic."""
    preset = {
        "name": "served",
        "velocity_thresh": 0.02,
        "merge_gap": 2.0,
        "min_window_duration": 2.0,
        "max_window_duration": 6.0,
        "hard_cap": True,
        "inactivity_end": 1.5,
        "inactivity_rally_thresh": 0.2,
        "inactivity_min_density": 0.3,
        "blob_fallback": True,
        "blob_factor": 4.0,
    }
    row = {
        "name": "vidX",
        "f1": 0.8, "f1@0.25": 0.7, "f1@0.5": 0.6,
        "merges": 2, "merge_rate": 0.5, "segment_count_ratio": 1.25,
        "num_pred": 5, "num_gt": 4,
        "tau_start": 2.0, "tau_end": 1.5,
        "tol_f1": 0.75, "tol_precision": 0.66, "tol_recall": 0.88,
        "dstart_abs_median": 0.40, "dend_abs_median": 1.20,
        "split_count": 1, "merge_count": 2,
    }

```

```

agg = {
    "n": 1,
    "f1": {"mean": 0.8, "ci_low": 0.8, "ci_high": 0.8},
    "f1@0.25": {"mean": 0.7, "ci_low": 0.7, "ci_high": 0.7},
    "f1@0.5": {"mean": 0.6, "ci_low": 0.6, "ci_high": 0.6},
    "merge_rate": {"mean": 0.5, "ci_low": 0.5, "ci_high": 0.5},
    "segment_count_ratio": {"mean": 1.25, "ci_low": 1.25, "ci_high": 1.25},
    "tol_f1": {"mean": 0.75, "ci_low": 0.75, "ci_high": 0.75},
    "tol_precision": {"mean": 0.66, "ci_low": 0.66, "ci_high": 0.66},
    "tol_recall": {"mean": 0.88, "ci_low": 0.88, "ci_high": 0.88},
    "dstart_abs_median": {"mean": 0.40, "ci_low": 0.40, "ci_high": 0.40},
    "dend_abs_median": {"mean": 1.20, "ci_low": 1.20, "ci_high": 1.20},
}
return {"preset": preset, "iou": 0.5, "min_crossings": 0, "rows": [row], "aggregate": agg}

def test_format_report_full_render():
    out = format_report(_result_fixture())
    expected = "\n".join([
        "# Rally-segmentation eval ? windowing 'served' (vt=0.02, merge_gap=2.0, "
        "min_win=2.0, cap=6.0(hard), inact_end=1.5/rt=0.2/md=0.3, blob_fallback), IoU=0.5",
        "",
        "| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio | pred/gt |",
        "|---|---|---|---|---|---|---|---|",
        "| vidX | 0.800 | 0.700 | 0.600 | 2 | 0.50 | 1.25 | 5/4 |",
        "",
        "***Aggregate (n=1, mean [95% CI]):** F1 0.800 [0.800,0.800] . "
        "F1@0.25 0.700 [0.700,0.700] . F1@0.5 0.600 [0.600,0.600] . "
        "merge_rate 0.500 [0.500,0.500] . seg_ratio 1.250 [1.250,1.250]",
        "",
        "## R2 ? tolerance-match (?_start=2.0s, ?_end=1.5s, strict 1:1)",
        "",
        "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
        "|---|---|---|---|---|---|---|---|",
        "| vidX | 0.750 | 0.66 | 0.88 | 0.40 | 1.20 | 1 | 2 |",
        "",
        "***R2 aggregate (n=1):** tolF1 0.750 [0.750,0.750] . tolP 0.660 [0.660,0.660] . "
        "tolR 0.880 [0.880,0.880] . |?start| 0.400 [0.400,0.400] . "
        "|?end| 1.200 [1.200,1.200] (start?solved / end=the gap)",
    ])
    assert out == expected

def test_format_report_no_bounded_knobs_omits_extras():
    result = _result_fixture()
    result["preset"] = {
        "name": "proposal",
        "velocity_thresh": 0.005,
        "merge_gap": 1.0,
        "min_window_duration": 0.5,
        "max_window_duration": 0.0,
        "hard_cap": False,
        "inactivity_end": 0.0,
        "inactivity_rally_thresh": 0.08,
        "inactivity_min_density": 0.34,
        "blob_fallback": False,
        "blob_factor": 4.0,
    }
    header = format_report(result).splitlines()[0]
    assert header == (
        "# Rally-segmentation eval ? windowing 'proposal' (vt=0.005, merge_gap=1.0, "
        "min_win=0.5), IoU=0.5"
    )

```

```

----- #
main() ? the dataclasses.replace preset-override wiring (read back via --json-out)
----- #

def _write_oscillating_traj(path, fps=30, secs=4, width=1280):
    """A fast shuttle oscillating across the net ? a stable window forms (mirrors the
    helper in test_rally_seg_eval)."""
    lo, hi, period = 200, width - 200, 20
    rows = ["Frame,Visibility,X,Y"]
    for i in range(int(fps * secs)):
        phase = (i % period) / period
        tri = 2 * phase if phase < 0.5 else 2 * (1 - phase)
        rows.append(f"{i},1,{lo + (hi - lo) * tri:.1f},300")
    path.write_text("\n".join(rows))
    return str(path)

def _golden_fixture(tmp_path):
    """Write a 1-video manifest + golden-features JSON; return (manifest_path, features_dir)."""
    traj = _write_oscillating_traj(tmp_path / "osc.csv")
    manifest = [{"name": "osc", "trajectory": traj, "fps": 30.0, "frame_width": 1280.0}]
    manifest_path = tmp_path / "manifest.json"
    manifest_path.write_text(json.dumps(manifest))
    (tmp_path / "osc_golden_features.json").write_text(json.dumps({"gts": [[0.0, 4.0]]}))
    return str(manifest_path), str(tmp_path)

def _run_main(monkeypatch, tmp_path, extra_argv):
    manifest_path, features_dir = _golden_fixture(tmp_path)
    json_out = tmp_path / "out.json"
    argv = ["rally_seg_eval", "--manifest", manifest_path, "--features-dir", features_dir,
            "--json-out", str(json_out), *extra_argv]
    monkeypatch.setattr("sys.argv", argv)
    main()
    with open(json_out) as fh:
        return json.load(fh)

def test_main_single_preset_writes_resolved_preset(monkeypatch, tmp_path, capsys):
    result = _run_main(monkeypatch, tmp_path, ["--preset", "proposal"])
    # non-vs path ? evaluate() result carries the resolved preset under "preset"
    assert result["preset"]["name"] == "proposal"
    assert result["preset"]["max_window_duration"] == 0.0
    assert result["preset"]["hard_cap"] is False
    assert result["preset"]["blob_fallback"] is False
    assert result["preset"]["inactivity_end"] == 0.0
    assert result["iou"] == 0.5 and result["min_crossings"] == 0
    out = capsys.readouterr().out
    assert "Rally-segmentation eval ? windowing 'proposal'" in out

def test_main_overrides_reach_base_preset(monkeypatch, tmp_path):
    result = _run_main(monkeypatch, tmp_path, [
        "--preset", "proposal",
        "--max-window-duration", "12",
        "--hard-cap",
        "--blob-fallback",
        "--blob-factor", "3.5",
        "--inactivity-end", "2.0",
        "--inactivity-rally-thresh", "0.11",
        "--inactivity-min-density", "0.4",
    ])
    p = result["preset"]
    assert p["max_window_duration"] == 12.0

```

```

assert p["hard_cap"] is True
assert p["blob_fallback"] is True
assert p["blob_factor"] == 3.5
assert p["inactivity_end"] == 2.0
assert p["inactivity_rally_thresh"] == 0.11
assert p["inactivity_min_density"] == 0.4

def test_main_vs_default_mirrors_base_overrides(monkeypatch, tmp_path):
    # --vs WITHOUT --vs-* variants ? cand mirrors base's cap/hard_cap/blob/inactivity overrides
    result = _run_main(monkeypatch, tmp_path, [
        "--preset", "proposal", "--vs", "proposal",
        "--max-window-duration", "10",
        "--hard-cap",
        "--blob-fallback",
        "--inactivity-end", "1.0",
    ])
    base, cand = result["base"]["preset"], result["cand"]["preset"]
    for p in (base, cand):
        assert p["max_window_duration"] == 10.0
        assert p["hard_cap"] is True
        assert p["blob_fallback"] is True
        assert p["inactivity_end"] == 1.0
    assert result["n_paired"] == 1

def test_main_vs_variants_override_only_cand(monkeypatch, tmp_path):
    # --vs-* variants let cand differ from base in one shot (baseline-uncapped vs candidate-
    capped)
    result = _run_main(monkeypatch, tmp_path, [
        "--preset", "proposal", "--vs", "proposal",
        "--vs-max-window-duration", "8",
        "--vs-hard-cap",
        "--vs-blob-fallback",
        "--vs-inactivity-end", "1.5",
    ])
    base, cand = result["base"]["preset"], result["cand"]["preset"]
    # base untouched (no base-side override given)
    assert base["max_window_duration"] == 0.0
    assert base["hard_cap"] is False
    assert base["blob_fallback"] is False
    assert base["inactivity_end"] == 0.0
    # cand carries the --vs-* overrides
    assert cand["max_window_duration"] == 8.0
    assert cand["hard_cap"] is True
    assert cand["blob_fallback"] is True
    assert cand["inactivity_end"] == 1.5

def test_main_unknown_preset_exits(monkeypatch, tmp_path):
    manifest_path, features_dir = _golden_fixture(tmp_path)
    argv = ["rally_seg_eval", "--manifest", manifest_path, "--features-dir", features_dir,
        "--preset", "nope"]
    monkeypatch.setattr("sys.argv", argv)
    with pytest.raises(SystemExit):
        main()

```

23. user (2026-06-18T10:22:49.779Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_1d39e81c-851-6\tests\test_rally_seg_eval_report.py (file state is current in your context ? no need to Read it back)

24. assistant (2026-06-18T10:22:52.080Z)

Now let me run these characterization tests against the UNCHANGED code to confirm they pass.

25. user (2026-06-18T10:22:55.898Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_1d39e81c-851-6
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 11 items

tests/test_rally_seg_eval_report.py::test_ci_none_renders_dash PASSED [ 9%]
tests/test_rally_seg_eval_report.py::test_ci_formats_mean_and_bounds_to_three_places PASSED [ 18%]
tests/test_rally_seg_eval_report.py::test_format_guard_pass_no_regress PASSED [ 27%]
tests/test_rally_seg_eval_report.py::test_format_guard_block_with_merge_rate_regress PASSED [ 36%]
tests/test_rally_seg_eval_report.py::test_format_report_full_render PASSED [ 45%]
tests/test_rally_seg_eval_report.py::test_format_report_no_bounded_knobs_omits_extras PASSED [ 54%]
tests/test_rally_seg_eval_report.py::test_main_single_preset_writes_resolved_preset PASSED [ 63%]
tests/test_rally_seg_eval_report.py::test_main_overrides_reach_base_preset PASSED [ 72%]
tests/test_rally_seg_eval_report.py::test_main_vs_default_mirrors_base_overrides PASSED [ 81%]
tests/test_rally_seg_eval_report.py::test_main_vs_variants_override_only_cand PASSED [ 90%]
tests/test_rally_seg_eval_report.py::test_main_unknown_preset_exits PASSED [100%]

===== 11 passed in 1.41s =====
```

26. assistant (2026-06-18T10:23:07.158Z)

All 11 characterization tests pass on the unchanged code. They pin:

- `\_ci` both branches, `\_format\_guard` both branches (pass + block-with-regress),
- `format\_report` full render + the bounded-knobs-omitted header,
- `main()` single-preset, base overrides reach base, `--vs` default mirrors base, `--vs-\*` variants override only cand, and the unknown-preset exit.

Now the refactor: extract `\_build\_preset(args, side)` (side ? {"base","cand"}) that returns the resolved preset, replacing the duplicated base/cand `--vs` wiring. I must keep outputs byte-identical. Let me re-read the exact wiring carefully to preserve order and semantics.

Key observations of the current logic:

- `\_apply\_inactivity(preset, end\_val)`: applies `inactivity\_end=end\_val` (if `end\_val` is not None), then always applies `inactivity\_rally\_thresh`/`inactivity\_min\_density` if those args are not None.
- Base side: cap = `args.max\_window\_duration`; hard_cap if `args.hard\_cap`; blob_fallback if `args.blob\_fallback`; blob_factor if `args.blob\_factor` is not None; then `\_apply\_inactivity(base, args.inactivity\_end)`.
- Cand side: cap = `vs\_max\_window\_duration` if not None else `max\_window\_duration`; hard_cap if `vs\_hard\_cap` or `hard\_cap`; blob_fallback if `vs\_blob\_fallback` or `blob\_fallback`; blob_factor if `args.blob\_factor` is not None; then `\_apply\_inactivity(cand, vs\_inactivity\_end` if not None else `inactivity\_end)`.

I'll write `\_build\_preset(args, side)` that resolves the preset name, cap, hard_cap, blob_fallback, blob_factor, and inactivity per side. Note: base resolves `args.preset`; cand resolves `args.vs`. The inactivity helper is shared identically across both sides (the rally_thresh/min_density apply to both unconditionally), so I'll fold it in.

Let me write the refactor.

27. user (2026-06-18T10:23:15.477Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-6\backend\eval\rally_seg_eval.py has been updated successfully. (file state is current in your context ? no need to Read it back)

28. assistant (2026-06-18T10:23:22.873Z)

I need to verify a subtle ordering difference. In the original, base side applies in this order: cap, hard_cap, blob_fallback, blob_factor, then `\_apply\_inactivity` (which does inactivity_end, rally_thresh, min_density). My `\_build\_preset` preserves exactly that order.

For the cand side, original order: cap, hard_cap, blob_fallback, blob_factor, then `\_apply\_inactivity`. My function preserves that too. Since `replace` on a frozen dataclass is order-independent for distinct fields (each touches a different field), ordering doesn't affect output, but I've kept it identical anyway.

Now update `main()` to use `\_build\_preset` and remove the duplicated wiring.

29. user (2026-06-18T10:23:27.463Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-6\backend\eval\rally_seg_eval.py has been updated successfully. (file state is current in your context ? no need to Read it back)

30. assistant (2026-06-18T10:23:31.619Z)

Let me verify the edited `main()` region reads cleanly.

31. user (2026-06-18T10:23:32.667Z)

```
300         preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
301     return preset
302
303
304     def main() -> None:
305         ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
306 (offline).")
307         ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
308         ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
309         ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
310 served)")
311         ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
312         ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
313         ap.add_argument("--min-crossings", type=int, default=0,
314             help="net-crossings rally-state gate on --preset (0=off; W2
315 Gate-A)")
316         ap.add_argument("--vs-min-crossings", type=int, default=0,
317             help="net-crossings gate on --vs (default: same as --min-
318 crossings)")
319         ap.add_argument("--max-window-duration", type=float, default=None,
320             help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
321 12)")
322         ap.add_argument("--vs-max-window-duration", type=float, default=None,
323             help="cap (s) on --vs ONLY (default: same as --max-window-duration);
324 lets a "
325             "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
326 one shot")
327         ap.add_argument("--hard-cap", action="store_true",
328             help="enforce the cap as a HARD cap on --preset (chop continuously-
329 active "
330             "windows at the cap when no inactivity gap exists; W1 hard-cap
331 fallback)")
332         ap.add_argument("--vs-hard-cap", action="store_true",
333             help="hard-cap fallback on --vs (default: same as --hard-cap)")
334         ap.add_argument("--blob-fallback", action="store_true",
```

```

326             help="R5 blob-fallback on --preset (after R4, force-chop any group
still over the "
327             "cap at its midpoint + flag/log it; the continuously-active
blob last resort)")
328         ap.add_argument("--vs-blob-fallback", action="store_true",
329             help="R5 blob-fallback on --vs (default: same as --blob-fallback)")
330         ap.add_argument("--blob-factor", type=float, default=None,
331             help="R5 magnitude gate (both presets): chop only groups > this ×
cap (default 4.0)")
332         ap.add_argument("--inactivity-end", type=float, default=None,
333             help="R4 rally-end inactivity trim (s) on --preset (0=off; trims
post-rally drift tail)")
334         ap.add_argument("--vs-inactivity-end", type=float, default=None,
335             help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
336         ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
337             help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
338         ap.add_argument("--inactivity-min-density", type=float, default=None,
339             help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
340         ap.add_argument("--tau-start", type=float, default=None,
341             help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
342         ap.add_argument("--tau-end", type=float, default=None,
343             help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
344         ap.add_argument("--guard-weight-merge", type=float, default=None,
345             help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
346         ap.add_argument("--guard-weight-split", type=float, default=None,
347             help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
348         ap.add_argument("--guard-counts", action="store_true",
349             help="R1 over-seg guard scores raw counts instead of normalized
rates "
350             "(rates are the default ? comparable across a structural
mega?bounded change)")
351         ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
352         args = ap.parse_args()
353
354         golden = load_golden(args.manifest, args.features_dir)
355         base = _build_preset(args, "base")
356         if args.vs:
357             vs_mc = args.vs_min_crossings or args.min_crossings
358             cand = _build_preset(args, "cand")
359             result = compare(golden, base, cand, args.iou,

```

32. assistant (2026-06-18T10:23:37.177Z)

The refactor looks clean. One subtle behavior difference to verify: in the original, when `--vs` is NOT given, the cand-side wiring never runs, so the cand-side `\_resolve\_preset(args.vs)` with `args.vs=None` was never reached. In my refactor, `\_build\_preset(args, "cand")` is only called inside the `if args.vs:` block, so `args.vs` is always truthy when "cand" is built. Good ? no behavior change for the no-`--vs` path.

Now re-verify the SAME characterization tests pass unchanged.

33. user (2026-06-18T10:23:40.700Z)

```

.....
18 passed in 1.52s

```

[100%]

34. assistant (2026-06-18T10:23:44.455Z)

All 18 pass (11 new characterization + 7 existing). Now run the full gate. First, capture base coverage isn't strictly needed since the task states base ~80.9%; I'll run the four gate commands and confirm coverage stays at/above 70 floor and doesn't drop vs base. Let me run the full suite first.

35. user (2026-06-18T10:24:53.477Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 8%]
..... [ 16%]
..... [ 24%]
..... [ 32%]
..... [ 41%]
..... [ 49%]
..... [ 57%]
..... [ 65%]
..... [ 74%]
.....SSSSS..... [ 82%]
..... [ 90%]
..... [ 98%]
..... [100%]
868 passed, 7 skipped in 65.78s (0:01:05)
=====
RESULT: PASS - all tests green
=====
```

36. assistant (2026-06-18T10:24:55.983Z)

Suite green: 868 passed, 7 skipped. Now ruff and mypy.

37. user (2026-06-18T10:24:57.312Z)

```
F401 [*] `backend.eval.rally_seg_eval` imported but unused
--> tests\test_rally_seg_eval_report.py:17:26
    |
15 | import pytest
16 |
17 | from backend.eval import rally_seg_eval
    |                               ^^^^^^^^^^^^^^^^^
18 | from backend.eval.rally_seg_eval import (
19 |     _ci,
    |
help: Remove unused import: `backend.eval.rally_seg_eval`

Found 1 error.
[*] 1 fixable with the `--fix` option.
```

38. user (2026-06-18T10:25:35.269Z)

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\_init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\_init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 107 source files
```

39. assistant (2026-06-18T10:25:37.984Z)

Mypy is clean. Ruff flagged an unused import in my test file ? I imported `rally\_seg\_eval` (the module) but only use the named imports. Let me remove that unused import.

40. user (2026-06-18T10:25:40.552Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\workspace\worktrees\wf_1d39e81c-851-6\tests\test_rally_seg_eval_report.py has been updated successfully. (file state is current in your context ? no need to Read it back)

41. user (2026-06-18T10:25:45.418Z)

All checks passed!

42. assistant (2026-06-18T10:25:48.271Z)

Ruff clean. Now run the coverage gate.

43. user (2026-06-18T10:27:32.438Z)

backend\reporting\run_signals.py	14	0	100%
backend\results.py	27	0	100%
backend\sports__init__.py	17	1	94%
backend\sports\badminton.py	13	1	92%
backend\sports\base.py	12	3	75%
backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL	8092	1423	82%
Required test coverage of 70% reached. Total coverage: 82.41%			
868 passed, 7 skipped, 7 warnings in 99.37s (0:01:39)			
=====			
RESULT: PASS - all tests green			
=====			

44. assistant (2026-06-18T10:27:35.672Z)

Total coverage 82.41% ? above the 70 floor and above the ~80.9% base, so no regression. Let me check the specific coverage of the file I touched to confirm the new tests actually exercised the report/main paths.

45. user (2026-06-18T10:27:41.666Z)

backend\eval\rally_seg_eval.py	192	4	98%	155, 187, 189, 386
TOTAL	192	4	98%	
18 passed in 2.48s				

46. assistant (2026-06-18T10:27:45.410Z)

98% coverage on the touched module ? the previously-zero-coverage `format\_report`/_format_guard/_ci/main() preset-override block is now exercised. The 4 missed lines (155/187/189 = lazy import lines inside functions; 386 = `if \_\_name\_\_` guard) are

incidental.

All four gates are green and coverage did not drop. Let me verify the diff and HEAD/branch right before committing, then stage explicit paths.

47. user (2026-06-18T10:27:48.296Z)

```
refactor/rally-seg-eval-build-preset
--- HEAD ---
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
--- origin/master..HEAD (should be empty until commit) ---
--- diffstat ---
  backend/eval/rally_seg_eval.py | 72 ++++++-----
  1 file changed, 39 insertions(+), 33 deletions(-)
--- untracked test ---
  M backend/eval/rally_seg_eval.py
  ?? tests/test_rally_seg_eval_report.py
```

48. assistant (2026-06-18T10:27:52.471Z)

Branch correct, `origin/master..HEAD` is empty (no sibling commits rode in), and only my two intended files changed. Let me verify the full diff of the production file to do a final byte-level sanity check that it's a pure move.

49. user (2026-06-18T10:27:53.138Z)

```
diff --git a/backend/eval/rally_seg_eval.py b/backend/eval/rally_seg_eval.py
index 7c0d52e..248e42d 100644
--- a/backend/eval/rally_seg_eval.py
+++ b/backend/eval/rally_seg_eval.py
@@ -264,6 +264,43 @@ def _resolve_preset(name: str) -> WindowingPreset:
     raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")

+def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
+    """Resolve the ``base`` (``--preset``) or ``cand`` (``--vs``) windowing from ``args`` by
+    applying the ``dataclasses.replace`` overrides for cap / hard-cap / blob-fallback /
+    inactivity. ``cand`` falls back to the base override when its ``--vs-`` variant is unset
+    (so a baseline-vs-candidate promotion check runs in one shot)."""
+    from dataclasses import replace
+    if side == "base":
+        preset = _resolve_preset(args.preset)
+        cap = args.max_window_duration
+        hard_cap = args.hard_cap
+        blob_fallback = args.blob_fallback
+        inactivity_end = args.inactivity_end
+    else:
+        preset = _resolve_preset(args.vs)
+        cap = (args.vs_max_window_duration if args.vs_max_window_duration is not None
+              else args.max_window_duration)
+        hard_cap = args.vs_hard_cap or args.hard_cap
+        blob_fallback = args.vs_blob_fallback or args.blob_fallback
+        inactivity_end = (args.vs_inactivity_end if args.vs_inactivity_end is not None
+                          else args.inactivity_end)
+    if cap is not None:
+        preset = replace(preset, max_window_duration=cap)
+    if hard_cap:
+        preset = replace(preset, hard_cap=True)
+    if blob_fallback:
+        preset = replace(preset, blob_fallback=True)
+    if args.blob_factor is not None:
+        preset = replace(preset, blob_factor=args.blob_factor)
+    if inactivity_end is not None:
```

```

+     preset = replace(preset, inactivity_end=inactivity_end)
+     if args.inactivity_rally_thresh is not None:
+         preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
+     if args.inactivity_min_density is not None:
+         preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
+     return preset
+
+
def main() -> None:
    ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
(offline).")
    ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
@@ -314,42 +351,11 @@ def main() -> None:
    ap.add_argument("--json-out", default=None, help="optional path to write the full result
JSON")
    args = ap.parse_args()

-     from dataclasses import replace
-     golden = load_golden(args.manifest, args.features_dir)
-     def _apply_inactivity(preset, end_val):
-         if end_val is not None:
-             preset = replace(preset, inactivity_end=end_val)
-         if args.inactivity_rally_thresh is not None:
-             preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
-         if args.inactivity_min_density is not None:
-             preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
-         return preset
-
-     base = _resolve_preset(args.preset)
-     if args.max_window_duration is not None:
-         base = replace(base, max_window_duration=args.max_window_duration)
-     if args.hard_cap:
-         base = replace(base, hard_cap=True)
-     if args.blob_fallback:
-         base = replace(base, blob_fallback=True)
-     if args.blob_factor is not None:
-         base = replace(base, blob_factor=args.blob_factor)
-     base = _apply_inactivity(base, args.inactivity_end)
+     base = _build_preset(args, "base")
    if args.vs:
        vs_mc = args.vs_min_crossings or args.min_crossings
        cand = _resolve_preset(args.vs)
        cand_cap = (args.vs_max_window_duration if args.vs_max_window_duration is not None
                    else args.max_window_duration)
        if cand_cap is not None:
            cand = replace(cand, max_window_duration=cand_cap)
        if args.vs_hard_cap or args.hard_cap:
            cand = replace(cand, hard_cap=True)
        if args.vs_blob_fallback or args.blob_fallback:
            cand = replace(cand, blob_fallback=True)
        if args.blob_factor is not None:
            cand = replace(cand, blob_factor=args.blob_factor)
        cand = _apply_inactivity(cand, args.vs_inactivity_end if args.vs_inactivity_end is not
None
-                                     else args.inactivity_end)
+     cand = _build_preset(args, "cand")
    result = compare(golden, base, cand, args.iou,
                    base_min_crossings=args.min_crossings, cand_min_crossings=vs_mc,
                    tau_start=args.tau_start, tau_end=args.tau_end,

```

50. assistant (2026-06-18T10:27:57.917Z)

This is a clean pure-move extraction: the duplicated base/cand `dataclasses.replace` wiring (including the inner `\_apply\_inactivity` closure) is consolidated into `\_build\_preset(args, side)`. Logic, override order, fallback semantics, and the `replace` field operations are all

identical. No thresholds, constants, log strings, or return types changed.

Now let me commit with explicit paths and push, then open the PR.

51. user (2026-06-18T10:28:00.022Z)

```
warning: in the working copy of 'tests/test_rally_seg_eval_report.py', LF will be replaced by
CRLF the next time Git touches it
M  backend/eval/rally_seg_eval.py
A  tests/test_rally_seg_eval_report.py
```

52. assistant (2026-06-18T10:28:03.128Z)

Only the two intended files staged. Let me re-verify HEAD hasn't shifted (concurrent-session safety), then commit.

53. user (2026-06-18T10:28:08.970Z)

```
--- branch ---
refactor/rally-seg-eval-build-preset
[refactor/rally-seg-eval-build-preset 0a014ba] refactor(eval): extract _build_preset(args, side)
from rally_seg_eval.main()
 2 files changed, 345 insertions(+), 33 deletions(-)
 create mode 100644 tests/test_rally_seg_eval_report.py
```

54. assistant (2026-06-18T10:28:11.383Z)

The `origin/master..HEAD` was empty before commit (confirming only my commit will land). Now push and open the PR.

55. user (2026-06-18T10:28:13.929Z)

```
remote:
remote: Create a pull request for 'refactor/rally-seg-eval-build-preset' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/rally-
seg-eval-build-preset
remote:
branch 'refactor/rally-seg-eval-build-preset' set up to track 'origin/refactor/rally-seg-eval-
build-preset'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      refactor/rally-seg-eval-build-preset -> refactor/rally-seg-eval-build-
preset
```

56. user (2026-06-18T10:28:32.214Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/231>

57. assistant (2026-06-18T10:28:34.687Z)

PR opened: <https://github.com/avidullu/badminton-highlight-indexer/pull/231>

All four gates green, coverage up (no regression), and the refactor is a verified pure move. Returning the structured result.

58. user (2026-06-18T10:28:53.097Z)

Structured output provided successfully

59. assistant (2026-06-18T10:29:02.690Z)

Done. Resolved the deferred tech-debt item `rally-seg-eval-report-extract` following the characterization-test-first discipline.

Summary:

- **Branch:** `refactor/rally-seg-eval-build-preset` (cut from `origin/master`, verified HEAD matched).
- **Characterization tests first:** added `tests/test\_rally\_seg\_eval\_report.py` (11 tests) pinning the previously zero-coverage `\_ci` / `\_format\_guard` / `format\_report` rendered strings and the `main()` preset-override wiring (driven via monkeypatched `sys.argv` on a synthetic golden fixture, read back through `--json-out`). Confirmed all 11 pass on the **unchanged** code.
- **Refactor (pure move):** extracted `\_build\_preset(args, side)` consolidating the duplicated base/cand `dataclasses.replace` wiring; the same 11 tests pass unchanged after.
- **Gate (all green):** suite 868 passed / 7 skipped; ruff clean; mypy clean; coverage 82.41% (? 70 floor, no regression vs ~80.9% base; touched module 0%?98%).
- **PR:** <https://github.com/avidullu/badminton-highlight-indexer/pull/231> (opened, not merged).

Files: `C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf\_1d39e81c-851-6\backend\eval\rally\_seg\_eval.py` and `...\tests\test\_rally\_seg\_eval\_report.py`.